

Agentic AI Engineering with Python

The 3-Month Pipeline to Build, Secure and Deploy
Production AI Agents

OpenAI

LangChain

LangGraph

CrewAI

MCP + A2A

AWS / GCP

01

PROBLEM

Map the business problem

02

AGENT

Reason, tools, memory

03

TOOLS

MCP, APIs, RAG

04

DEPLOY

Secure, evaluate, ship

Start 16 August 2026 · Sat–Sun 8:00–11:00 PM IST · 3 months

Core Python required · No prior AI experience · Live + recorded

[STUDENT PLAYBOOK](#) · [FDE TRACK](#)

01 · WHY THIS COURSE EXISTS

Course vision and philosophy

This is not a theory-first course. Every concept is connected to working Python code, a practical project and a production decision. You move beyond basic LLM integration and learn how intelligent systems reason, use tools, remember context and complete real work.

You will build with the **OpenAI Agents SDK**, **LangChain**, **LangGraph**, **CrewAI**, **Model Context Protocol** and **Agent2Agent**. Every major stage ends with something you can run, test and deploy. Finish with a portfolio that proves you can take an AI system from idea to production.

The FDE delivery loop you will practice

A Forward Deployed Engineer works close to a real customer problem, designs the solution, builds it, deploys it and owns it in production.

Step	Loop	What you practice
1	Discover	Understand the customer problem and domain constraints.
2	Scope	Design the solution. Define success metrics and risk.
3	Build	Agent + tools + memory + guardrails + evaluation dataset.
4	Ship	Secure, deploy, authenticate, stream, monitor.
5	Own	Measure, iterate, troubleshoot and hand off a live system.

Course essentials

Item	Detail
Start date	16 August 2026
Batch	Weekend live — Saturday and Sunday
Time	8:00 PM to 11:00 PM IST (3 hours)
Duration	3 months · live + recorded
Prerequisite	Core Python. No prior AI experience required.
Included refreshers	Core Python, NLP basics, LLM foundations, ML intuition, developer tooling
Contact	+91 90089 63671 · WhatsApp or call
Refund window	Available until 31 August 2026 (official repo)
Code repo	github.com/telusko-aliens/agentic-ai-engineering-with-python
Enrollment	go.telusko.com/agentic-ai-python · coupon TELUSKO10

02 · OUTCOMES, NOT TOPICS

Skill map — what you will be able to do

The course combines model fundamentals, application engineering, agent orchestration, security, evaluation and cloud delivery. Use this as a self-check at the end of each stage.

01 Chat APIs & Agents SDK

Build with Chat APIs, Responses API and the OpenAI Agents SDK.

02 Model internals

Explain transformers, tokens, embeddings and context windows.

03 Cloud delivery

Deploy with Docker, GitHub Actions, AWS and Google Cloud.

04 Security

Defend against prompt injection, tool misuse and data leakage.

05 Multi-agent

Orchestrate systems with LangGraph and CrewAI.

06 Streaming APIs

Serve streaming agent APIs with FastAPI, SSE and WebSockets.

07 Production RAG

Design RAG with hybrid search, filters and reranking.

08 LangChain apps

Build tools, agents and middleware with LangChain.

09 FDE cycle

Own the complete Forward Deployed Engineer delivery cycle.

10 Interfaces

Build UIs with Streamlit, Chainlit, Gradio, React or Next.js.

11 Structured I/O

Create reliable structured outputs with Pydantic.

12 Interop

Connect tools and agents through MCP and A2A.

13 Document AI

Ingest scanned PDFs, tables, charts and page-level metadata.

14 Prompting craft

Use few-shot prompting, decomposition, ReAct and reflection.

15 Evaluation

Evaluate groundedness, tool choice, trajectories and safety.

16 Control planes

Create persistent state graphs and human approval flows.

03 · DEPLOYABLE ARTEFACTS

Project portfolio — build proof, not notes

Each project is framed around a real customer scenario and adds a deployable piece to your portfolio. The capstone takes one customer problem from discovery to a live, monitored deployment.

#	Project	Stack signal	FDE scenario
M2	Intelligent Document Extraction	OpenAI + Pydantic	Automate a client's manual data-entry workflow
M3	Tool-Enabled Customer Support	LangChain + tools + memory	Cut a client's support-ticket volume
M4	Secure Enterprise RAG Assistant	Hybrid retrieval + citations	Internal ask-anything assistant
M5	Research & Report Agent	LangGraph + HITL + memory	Automate research reporting
M6	Multi-Agent Customer Onboarding	CrewAI Flows + guardrails	Onboard a client's users end-to-end
M7	Cross-Agent Workflow	A2A + secured MCP tools	Integrate agents with client systems
M8	Hardening an Agent for Production	Guardrails, audit, isolation	Pass a client's security review
M9	Evaluation & Monitoring Harness	LangSmith + CI eval gate	Prove quality and ROI
M10	Full-Stack Agent App	FastAPI + streaming UI	A usable product for end-users
M11	Ship to AWS Bedrock AgentCore	Docker + CI/CD + cloud	A real production launch
M12	FDE Capstone	Chosen customer problem	Discover → scope → build → deploy → own

Capstone delivery standard

Every capstone must include: architecture document and GitHub repository · automated tests and evaluation dataset · security checklist and live deployment URL · monitoring dashboard and recorded demo. Choose one track: Enterprise Knowledge & Support Agent · Analytics Agent over BigQuery/SQL · Operations Agent with email/CRM/calendar tools, OAuth, approvals, MCP/A2A.

04 · TWELVE MODULES, ONE PIPELINE

Six-stage learning roadmap

Stage	Modules	Name	You leave with
01 Foundations	M1	Python, NLP, LLMs, prompting	Working mental model of agents vs chat apps
02 Build	M2–M4	OpenAI, LangChain, RAG	Extraction service + support app + enterprise RAG
03 Orchestrate	M5–M6	LangGraph and CrewAI	Stateful research agent + onboarding crew
04 Connect	M7	MCP and Agent2Agent	Cross-system agent collaboration
05 Harden	M8–M9	Security, eval, observability	Red-teamed agent + CI quality gate
06 Ship	M10–M12	APIs, UI, cloud, FDE capstone	Live monitored product + portfolio

Recorded support is included. Live coding runs throughout weekend sessions. Doubt-solving happens after class. Projects stay tied to business outcomes.

05 · FOUNDATIONS THROUGH ORCHESTRATION

Module playbooks M1–M6

M1 · RECORDED — Agentic AI and Python Foundations

Build the technical base before the live projects begin.

Getting started with Agentic AI

What Agentic AI is and why it matters now. Agents vs LLM chat apps vs generative AI. The core agent loop: reasoning, planning, memory, tools. Industry use cases and the FDE mindset. Frameworks you will master: OpenAI SDK, LangChain, LangGraph, CrewAI.

Python for AI engineering

Prerequisite: functions, classes, exceptions, modules, files. Type hints, dataclasses and Pydantic-ready code. async/await, generators, decorators, context managers. Project setup with uv, virtualenvs and dependencies. Secrets, .env, HTTP & JSON, basic SQL, Git and the command line.

NLP and LLM essentials

Tokenization and embeddings. Transformers and attention as working intuition. Tokens, context windows and pricing. Temperature, top-p, sampling and determinism. Hallucination, chat roles and the tool-calling concept.

Prompt engineering foundations

Zero-shot, few-shot and task decomposition. Role prompting and reflection. Reasoning-model prompting. Structured-output prompting and reusable templates. Programmatic prompt optimization with DSPy (intro).

Project / exit artefact

No live project. Exit check: you can set up a uv project, call an LLM, explain tokens/cost, and write a structured prompt template.

M2 · LIVE PROJECT — LLM APIs, Responses API & Agents SDK

Move from raw model calls to reliable, tool-enabled AI services.

Working with LLM APIs

OpenAI SDK: chat, streaming and parameters. Anthropic, Gemini and Ollama. Multimodal inputs. Token usage, cost control and model selection.

Structured outputs with Pydantic

Models, fields and validators. JSON mode vs schema-enforced Structured Outputs. Nested models. Handling refusals, validation failures and retries.

OpenAI Responses API & Agents SDK

Unified agent interface. Stateful multi-turn and background mode. Hosted tools: web search, file search, Code Interpreter. Function calling and remote MCP tools. Agents, tools, handoffs, sessions, guardrails, tracing.

Evaluation from day one

Why evaluation starts now. Small evaluation dataset and baseline. Auto-validating structured outputs and correctness checks.

Project / exit artefact

Intelligent Document Extraction Service — invoices, resumes and emails. Schema validation, retries and confidence scoring. FDE: automate a client's manual data-entry workflow.

M3 · LIVE PROJECT — LangChain

Build flexible applications that combine models, tools, documents and memory.

Fundamentals and LCEL

Chat models, messages and multimodal across providers. Prompt templates and output parsers. LCEL Runnables: pipe, parallel, passthrough. Streaming and batching. Ollama and Hugging Face.

Embeddings and vector stores

OpenAI, Gemini, Hugging Face, Ollama embeddings. Text splitters and retrieval strategies. FAISS or Chroma locally; pgvector or Qdrant in production. Milvus and Pinecone compared. Basic RAG flow.

Tools and agents

Defining and validating custom tools. The tool-calling loop and ReAct. create_agent harness. Structured output and agent memory.

Middleware, ingestion

Dynamic prompts and tool selection. Retries and summarization middleware. Model routing. PII protection. Custom agent state. PDF, web, CSV, DB loaders. Scanned PDFs and OCR. Tables, forms, layout-aware chunking. Image/chart extraction and page citations.

Project / exit artefact

Tool-Enabled Customer Support Application — answers from docs + live APIs via tools. Conversation memory, middleware and graceful fallbacks. FDE: cut support-ticket volume.

M4 · LIVE PROJECT — RAG

Turn document search into a grounded, measurable and secure system.

RAG fundamentals

Indexing → retrieval → generation. Chunking strategies that actually work. Answer citations and source grounding. RAG evaluation: relevance, groundedness, citation accuracy.

Production RAG techniques

Query rewriting and multi-query retrieval. Hybrid search (keyword + vector). Metadata filtering. Reranking.

Advanced RAG

Corrective and adaptive RAG. Self-RAG. GraphRAG and knowledge-graph retrieval. Retrieval optimization with DSPy.

Project / exit artefact

Secure Enterprise RAG Assistant — self-correcting RAG over private company documents. Citations, freshness and access control. FDE: internal ask-anything assistant.

M5 · CORE PROJECT — LangGraph: Building Real Agents

Use explicit state, persistence and control to build agents that can recover, pause and collaborate.

LangGraph core

Why graphs beat linear chains. StateGraph: nodes, edges, conditional edges. State schema and reducers. Recursion limits and streaming modes (values / updates / messages).

Memory and persistence

Short-term memory: checkpoints and threads. Long-term memory store. Message trimming and summarization. SQLite / Postgres persistence.

Human-in-the-loop and control

Breakpoints and interrupts. Approving and editing agent state. Time-travel debugging. Durable execution and resume-after-failure.

Patterns and multi-agent

create_agent vs custom StateGraph. Tool nodes and routing. Parallel, conditional and iterative workflows. Supervisor and hierarchical teams. Network topology and handoffs. Subgraphs and shared state. LangChain Deep Agents. Context engineering.

Project / exit artefact

Research & Report Agent — plan → research (tools) → write → self-review. Long-term memory + human approval gate. FDE: automate a client's research reporting.

M6 · LIVE PROJECT — CrewAI: Multi-Agent Orchestration

Design role-based teams and combine flexible agents with deterministic production flows.

CrewAI introduction

Enterprise multi-agent orchestration. CrewAI vs LangGraph: when to pick which. crewai CLI. LiteLLM to any model. Your first Crew.

Agents, tasks, crews

Role, goal, backstory and tools. Task description, expected output and context. Sequential vs hierarchical process. Delegation and collaboration.

Tools, knowledge, memory

Built-in and custom tools. LangChain tool integration. Knowledge sources and RAG inside CrewAI. Short-term, long-term and entity memory. Guardrails and observability.

Flows for production

@start, @listen and @router. Typed serializable state with Pydantic. Branching and conditional logic. Combining Flows with Crews. Async execution and checkpointing. Enterprise deploy options.

Project / exit artefact

Multi-Agent Customer Onboarding Workflow — crew automating onboarding, Flow orchestration with typed state and guardrails. FDE: onboard a client's users end-to-end.

06 · CONNECT, HARDEN, SHIP

Module playbooks M7–M12

M7 · LIVE PROJECT — MCP and A2A Interoperability

Connect agents to external capabilities and let specialised agents work across system boundaries.

Model Context Protocol

How MCP connects AI applications to tools, data and external systems. Tools, resources and prompts. Transports: stdio and Streamable HTTP. Local and remote MCP servers. Tool discovery and error handling.

Agent2Agent protocol

A2A vs MCP: when to use each. Agent Cards and capability discovery. Messages, tasks and the task lifecycle. Artifacts and streaming. Agent-to-agent authentication and authorization.

Security and integration

OAuth and authorization. MCP security and malicious-tool defence. Custom MCP server for internal APIs. Using MCP tools with OpenAI, LangChain/LangGraph and CrewAI.

Project / exit artefact

Cross-Agent Workflow — two specialised agents collaborating over A2A, each using secured MCP tools from internal APIs. FDE: integrate agents with a client's systems.

M8 · LIVE PROJECT — Agent Security and Reliability

Treat agent actions, data and tools as production attack surfaces from the start.

Agent threats

Direct and indirect prompt injection. Agent goal hijacking. Tool misuse and malicious MCP tools. Memory poisoning and cross-user leakage.

Defences and controls

Tool argument validation and least-privilege permissions. Human approval before destructive actions. Agent identity and permissions. Output sanitisation and sandboxed code execution. PII redaction, audit logs and secrets handling.

Production reliability

Timeouts, retries and exponential backoff. Rate-limit handling and idempotent tools. Max steps, token and cost limits. Durable execution, queues and prompt/semantic caching. Health and readiness checks.

Project / exit artefact

Hardening an Agent for Production — add guardrails, approval, audit and isolation to an existing agent. Red-team it with prompt-injection attacks. FDE: pass a client's security review.

M9 · LIVE PROJECT — Evaluation and Observability

Measure quality, cost and reliability so production decisions are based on evidence.

Agent evaluation

Datasets and offline evaluation. Tool-selection accuracy and expected trajectory. Task-completion, groundedness and citation correctness. LLM-as-judge and its limitations. Regression, safety and adversarial tests. Cost and latency percentiles.

Tracing and monitoring

LangSmith for tracing and evaluation. Tracing runs, projects and the prompt hub. OpenTelemetry logs, metrics and traces. Production dashboards and alerts. Opik overview.

Project / exit artefact

Evaluation & Monitoring Harness — eval dataset + CI evaluation gate + live monitoring dashboard. FDE: prove quality and ROI to a client.

M10 · LIVE PROJECT — Production APIs & UI

Turn the agent into a usable product with streaming, authentication and clear human control.

Serving agents as APIs

FastAPI backend and endpoint design. SSE and WebSocket streaming. Authentication and secrets management. Session management and recovery.

Agent UX and human loops

Human-approval UI for sensitive actions. Tool-call status and step visualisation. Citations, feedback and auditability. Error and retry handling.

Building the frontend

Rapid UIs with Streamlit, Chainlit or Gradio. Production frontend with React / Next.js. Streaming responses and conversation management.

Project / exit artefact

Full-Stack Agent App — complete chat app with streaming, auth, approval and feedback UI. FDE: a usable product for a client's end-users.

M11 · LIVE PROJECT — AWS and Google Cloud Deployment

Containerize, automate releases and operate agents with production cloud services.

Containerization and CI/CD

Dockerizing agents. LangGraph Agent Server and LangSmith Deployment. GitHub Actions pipelines. Tests, offline evals and preview deployments in CI. Prompt/version management, staging, rollback and quality gates.

Deploying on AWS

AgentCore Memory, Gateway (APIs → MCP tools) and Identity. Amazon Bedrock models and Knowledge Bases. ECS/Fargate, Lambda, ECR, S3, API Gateway and CloudWatch. Bedrock AgentCore Runtime for serverless agent hosting.

Deploying on Google Cloud + self-host

Gemini Enterprise Agent Platform and Agent Runtime. BigQuery data agents and Vertex AI Vector Search. Cloud Run and GKE. Serving open models with vLLM or Hugging Face TGI. When to self-host. PEFT / LoRA for domain adaptation.

Project / exit artefact

Ship an Agent to AWS Bedrock AgentCore — containerize, deploy, live production endpoint. Add AgentCore memory, gateway tools and monitoring. FDE: a real production launch.

M12 · DEPLOYED CAPSTONE — The Forward Deployed Engineer Capstone

Bring discovery, engineering, security, evaluation and deployment together in one complete delivery.

The FDE delivery playbook

Discover: map the customer problem and domain. Scope: design the solution and define success metrics. Build: agent + tools + guardrails + evals. Deploy: production launch and handoff. Own: monitor, troubleshoot and iterate.

Choose one capstone track

Enterprise Knowledge & Support Agent (RAG, access control, escalation). Analytics Agent over BigQuery / SQL (NL → SQL, read-only, guardrails). Operations Agent (email / CRM / calendar tools, OAuth, approvals, MCP/A2A).

Delivery standard — every capstone

Architecture document and GitHub repository. Automated tests and evaluation dataset. Security checklist and live deployment URL. Monitoring dashboard and recorded demo.

Project / exit artefact

Take one customer problem from discovery to a live, monitored deployment. This is the portfolio piece that proves you can own an AI system after launch.

07 · STUDENT OPERATING SYSTEM

How to run the 3-month pipeline

Weekly cadence (Saturday + Sunday live)

When	Do this
Before class (2–4 hrs)	Watch recorded refreshers if needed. Skim last session code in the official repo. Write 3 questions.
During live session	Code along. Do not only watch. Commit a working snapshot before you leave the session.
Same night	Re-run the demo from a clean directory. Note every failure. Fix one of them.
Between weekends	Finish the module project to a deployable minimum. Add an eval case. Write a 10-line README of decisions.
End of each stage	Record a 3-minute demo. Push to GitHub. Update the portfolio table below.

Definition of done for every project

- Runs from a fresh clone with a documented `.env.example` and `uv/pip` lockfile.
- Has at least one structured output schema (Pydantic) and handles validation failure.
- Has a tiny eval set (10–30 cases) and a script that prints pass/fail.
- Logs or traces a run (LangSmith, OpenTelemetry, or structured logs).
- States threat assumptions: what a prompt-injection or tool-misuse attempt would look like.
- README names the customer problem, success metric, stack, and known limits.

Reference stack

OpenAI · LangChain · LangGraph · LangSmith · CrewAI · MCP · A2A · AWS Bedrock AgentCore · Google Cloud · FastAPI · Streamlit / Chainlit / Gradio / Next.js · Docker · GitHub Actions · OWASP guidance for LLM apps.

What you leave with

1	A portfolio of deployed projects, not notebook demos.
2	A complete FDE capstone with architecture, evals, security checklist and live URL.
3	Production security and evaluation habits you can defend in an interview or client review.
4	The confidence to own an AI system after launch — monitor, troubleshoot, iterate.

Learning experience and credentials

3 hours every Saturday and Sunday. Live coding throughout each session. Real use cases built from scratch. Recorded session access. Active peer learning community. Certificate of Completion — shareable on LinkedIn and your resume. A clear record that you completed the live learning path and production capstone.

Next step

Enroll or ask questions: go.telusko.com/agentic-ai-python · WhatsApp/call +91 90089 63671. Follow session folders in the official GitHub repo as classes land. Treat this playbook as the map; treat working, evaluated, deployed code as the proof.

Compiled from the Telusko Agentic AI Engineering with Python course brochure. Dates, policies and pricing should be confirmed on the official course page.